TechStore Pro

Informe técnico y evidencias de despliegue

Sistema de gestión de devoluciones integrado a una tienda tecnológica

Ficha del proyecto	Detalle
Actividad	Proyecto avanzado - Núcleo de Desarrollo de Software
Responsable de repositorios	Franz Aguilar / Fcranz28
Fecha de verificación	15 de octubre de 2026 - America/Lima
Frontend	Angular, desplegado en Vercel
Backend	Java 17 + Spring Boot, contenedor Docker en Render
Persistencia	PostgreSQL de Supabase; esquema privado techstore
Aplicación pública	https://techstorepro.vercel.app/

Adaptación del caso SoftDevoluciones sobre TechStore Pro. Se reutilizan usuarios, catálogo, compras y detalles existentes; el módulo de devoluciones incorpora validaciones, workflow y trazabilidad.

Este documento reúne implementación, correspondencia con la actividad, pruebas y capturas reales del despliegue. Las credenciales de infraestructura y los tokens no se incluyen.

1. Descripción y objetivos

Contexto y solución

TechStore Pro es una aplicación fullstack de accesorios tecnológicos. Sobre su proceso de compra se implementó un módulo posventa para centralizar solicitudes de devolución asociadas a compras reales de la aplicación, evitando cantidades excesivas, productos ajenos y cambios de estado arbitrarios.

Objetivo general

Gestionar devoluciones de compras existentes con autenticación JWT, autorización por rol, cálculo de importes en el servidor, validación de unidades disponibles y seguimiento auditable.

Objetivos específicos

• Permitir al cliente consultar compras, seleccionar líneas y cantidades y conocer el avance de sus solicitudes.

• Permitir al operador y administrador filtrar solicitudes, revisarlas, aprobarlas, rechazarlas con observación y completar las aprobadas.

• Mantener consistencia mediante transacciones, bloqueos y validaciones de pertenencia y disponibilidad.

• Publicar frontend y backend por separado y aportar repositorios, migraciones, colección Postman, pruebas y evidencias.

Alcance y diferencias con el caso base

La tienda y el checkout simulado son capacidades adicionales del proyecto existente. El núcleo evaluado es el módulo de devoluciones. No se ejecutan pagos ni reembolsos bancarios reales, ni transporte, facturación electrónica o notificaciones externas. Cloudinary almacena imágenes de productos, no fotografías adjuntas a devoluciones.

Guía de lectura

Secciones 2-4: arquitectura, datos, roles y workflow. Secciones 5-7: trazabilidad de requisitos. Secciones 8-11: API, pruebas, despliegue y ejecución. Secciones 12-16: capturas del navegador. Sección 17: conclusiones y pendientes.

2. Arquitectura y organización

Capa	Tecnología / responsabilidad
Presentación	Angular 21.2, TypeScript 5.9, componentes standalone, signals, RxJS, formularios reactivos en devoluciones y estilos CSS/Tailwind.
API y negocio	Spring Boot 4.1.1 y Java 17. Controladores REST, servicios transaccionales, DTOs y repositorios Spring Data JPA.
Seguridad	Spring Security con JWT HS256, validación de firma, emisor y vencimiento; BCrypt para contraseñas.
Persistencia	PostgreSQL en Supabase. Conexión JDBC SSL mediante Session pooler (puerto 5432).
Archivos	Cloudinary: subidas firmadas desde el backend; la URL HTTPS se conserva al guardar el producto.
Despliegue	Vercel sirve Angular; Render construye y ejecuta el Dockerfile; GitHub conserva dos repositorios.

Flujo de comunicación

Navegador → Angular/Vercel → HTTPS + JWT → API/Render → JDBC SSL → PostgreSQL/Supabase. Para imágenes: navegador → endpoint administrativo → Cloudinary; la API devuelve la URL segura.

Estructura del backend

Paquete raíz: **com.techstorepro.techstorepro**. Módulos: auth, usuario, producto, carrito, pedido, admin y devolucion. Cada módulo agrupa controller, service, repository, entity y DTO según su función. Las reglas y transiciones residen en los servicios, no en las vistas.

Estructura del frontend

core/: API, autenticación, guards, interceptores y almacenamiento del carrito. **features/**: inicio, catálogo, carrito/checkout, autenticación, pedidos, devoluciones y administración. **shared/**: visuales, progreso del pedido e historial. Los entornos separan localhost de la API HTTPS de producción.

3. Modelo relacional

Tabla	Campos y relaciones principales
usuarios	id (PK), nombre, email único, password_hash, rol, created_at.
productos	id (PK), nombre, descripción, categoría, precio, stock, imagen_url, destacado y activo.
carrito_items	id (PK), usuario_id → usuarios, producto_id → productos, cantidad.
pedidos	id (PK), usuario_id → usuarios, total, estado, created_at; datos de entrega y estado_pago para checkout simulado.
pedido_detalles	id (PK), pedido_id → pedidos, producto_id → productos, nombre y precio unitario histórico, cantidad y subtotal.
devoluciones	id (PK), pedido_id → pedidos, usuario_id → usuarios, estado, motivo, comentario, observación, importe y fechas.
devolucion_detalles	id (PK), devolucion_id → devoluciones, pedido_detalle_id → pedido_detalles, cantidad, precio_unitario y subtotal.
devolucion_historial	id (PK), devolucion_id → devoluciones, estado anterior/nuevo, actor_id → usuarios, nombre/rol del actor, observación y fecha.

Cardinalidades

Usuario 1:N pedidos y devoluciones. Pedido 1:N detalles y devoluciones. Devolución 1:N detalles e historial. Detalle de compra 1:N detalles de devolución. Producto 1:N detalles de compra y líneas de carrito.

Integridad y migraciones

Las claves foráneas preservan referencias; las restricciones controlan cantidades positivas, importes no negativos y estados válidos. Una línea de compra no puede repetirse dentro de una misma solicitud. Hay índices sobre propietario, estado, fechas y claves foráneas de consulta.

Flyway conserva V1-V11 para desarrollo. Supabase dispone de migraciones versionadas de bootstrap, índices, checkout e historial. El perfil remoto desactiva Flyway para no duplicar la historia ya aplicada y Hibernate valida el esquema techstore. El historial nuevo tiene RLS y acceso revocado a anon/authenticated; la autorización de la aplicación se ejecuta en la API.

4. Roles, workflow y reglas

Rol de la actividad	Implementación y permisos
CLIENTE	Se representa mediante USER. Consulta solo sus compras y solicitudes; registra devoluciones.
OPERADOR	Consulta gestión, inicia revisión, aprueba/rechaza y completa solicitudes.
ADMIN	Acceso a gestión global de devoluciones, productos, pedidos y resumen administrativo.

Flujo permitido

SOLICITADA → EN_REVISION → APROBADA → COMPLETADA
Alternativa: **EN_REVISION → RECHAZADA**. No hay retrocesos ni continuación desde estados finales. La API recibe estadoActual y devuelve HTTP 409 si la solicitud cambió durante la edición.

Regla	Aplicación en TechStore Pro
RN-01	Se comprueba que cada pedidoDetalleId pertenezca al pedido elegido.
RN-02	Cantidad mayor que cero y no superior a las unidades devolvibles.
RN-03	Se descuentan unidades de solicitudes APROBADAS o COMPLETADAS; se revalida al aprobar bajo bloqueo del pedido.
RN-04	La nueva solicitud inicia automáticamente en SOLICITADA con fecha de servidor.
RN-05	EstadoDevolucion determina únicamente las transiciones permitidas.
RN-06	RECHAZADA es final; una transición adicional se rechaza.
RN-07	Importe = suma(cantidad × precio unitario original de la compra); el cliente no fija el importe.
RN-08	El propietario se obtiene del JWT; compras y solicitudes ajenas devuelven 404.

Trazabilidad adicional

La creación y cada transición válida añaden un evento en la misma transacción. El actor se obtiene del JWT, no del cuerpo JSON. Se conserva nombre/rol del momento y observación; los intentos inválidos no generan movimientos. Para registros previos se identifica explícitamente que la historia anterior no está disponible.

5. Requisitos del backend

Código	Implementación / evidencia
RF-BE-01	POST /auth/login, Spring Security y emisor JWT techstore-pro.
RF-BE-02	SecurityConfig aplica USER, OPERADOR y ADMIN por ruta y método.
RF-BE-03	GET /devoluciones/compras lista compras del usuario autenticado.
RF-BE-04	GET /devoluciones/compras/{id} devuelve líneas, precios y unidades disponibles.
RF-BE-05	POST /devoluciones vincula la solicitud a un pedido existente y entregado.
RF-BE-06	El servicio valida el detalle de compra antes de persistir.
RF-BE-07	Valida cantidades, unidades aprobadas/completadas y revalida al aprobar.
RF-BE-08	Entidad y servicio asignan estado SOLICITADA y fecha del servidor.
RF-BE-09	PATCH /devoluciones/gestion/{id}/estado valida workflow y estado esperado.
RF-BE-10	RECHAZADA exige una observación no vacía.
RF-BE-11	Cálculo con BigDecimal y precio original de pedido_detalles.
RF-BE-12	Filtros estado, motivo, desde y hasta; hasta es exclusivo.
RF-BE-13	Parámetros pagina y tamanio en el listado de gestión.
RF-BE-14	DevolucionErrors usa @RestControllerAdvice para respuestas claras del módulo: 400, 404 y 409; seguridad aporta 401/403.

Organización no funcional

La API devuelve DTOs en lugar de entidades JPA, usa validación Jakarta y mantiene reglas en service. El origen CORS permitido es exactamente https://techstorepro.vercel.app; no se habilita indiscriminadamente *.vercel.app. Los valores secretos proceden de variables de entorno.

6. Requisitos del frontend

Código	Implementación / evidencia
RF-FE-01	Login y JWT de autenticación en sessionStorage; aviso al vencer la sesión.
RF-FE-02	authGuard, adminGuard y operatorGuard protegen navegación según identidad y rol.
RF-FE-03	Interceptor adjunta Bearer solo a la API propia; login/registro se excluyen.
RF-FE-04	Mis pedidos y Mis devoluciones consumen compras reales desde la API.
RF-FE-05	/devoluciones/nueva/:pedidoId muestra líneas y cantidades disponibles.
RF-FE-06	FormGroup/FormArray, ReactiveFormsModule y Validators para motivo, comentario y líneas.
RF-FE-07	Listado propio con estado, líneas, importe, observación e historial.
RF-FE-08	Panel OPERADOR/ADMIN con tarjetas, filtros y paginación; adaptación visual de la tabla sugerida.
RF-FE-09	Solo se muestran acciones válidas; rechazo requiere observación y controles se deshabilitan durante envío.
RF-FE-10	Mensajes de éxito/error, espera prolongada y reintento; no se reenvían automáticamente compras.

Interfaz y persistencia

Se utiliza una interfaz propia con CSS Grid/Flexbox y Tailwind, no Angular Material. El JWT de invitado se conserva en localStorage y migra selecciones antiguas de sessionStorage. Su firma y vigencia las valida el backend; no concede acceso a cuentas. Después del login se transfiere el carrito; el JWT de login no se persiste en localStorage.

Separación de funcionalidades

El pago simulado y el catálogo comercial amplían la tienda existente. El formulario de devolución no solicita evidencia fotográfica porque esa función está fuera del alcance obligatorio del enunciado. Las capturas distinguen estas capacidades.

7. Historias y aceptación

Historia	Correspondencia funcional
HU01	Inicio de sesión JWT y rutas protegidas.
HU02	Compras y detalle con productos y unidades.
HU03	Solicitud reactiva por líneas, cantidades y motivo.
HU04	Consulta propia con estado e historial.
HU05	Gestión de solicitudes pendientes.
HU06	Transición SOLICITADA → EN_REVISION.
HU07	Aprobación o rechazo desde revisión; observación obligatoria al rechazar.
HU08	APROBADA → COMPLETADA.
HU09	Filtros por estado, motivo y fecha.
HU10	Validaciones de cantidades, pertenencia y transiciones en el servidor.
HU11	Gestión global de compras y devoluciones, catálogo y datos de soporte desde ADMIN.

Criterios de aceptación 1-10

1-3: autenticación obligatoria, propiedad comprobada y validación en ambos extremos. **4-6:** control de unidades, rechazo de estados inválidos y cálculo con precios históricos. **7-8:** gestión paginada e integración Angular-Spring Boot con datos persistidos. **9:** pruebas unitarias y web MockMvc, junto con evidencia funcional en línea. **10:** URLs públicas y documentación de ejecución. La cobertura no equivale a una suite E2E automatizada contra PostgreSQL real: esa ampliación se identifica como pendiente.

8. API REST y ejemplos

Método / ruta	Acceso y propósito
POST /auth/register; /auth/login	Público. Registro de USER y autenticación.
GET /auth/me	Usuario autenticado. Identidad actual.
GET /devoluciones/compras	USER/ADMIN. Compras propias.
GET /devoluciones/compras/{id}	USER/ADMIN. Compra propia y líneas.
POST /devoluciones	USER/ADMIN. Crear solicitud.
GET /devoluciones; /devoluciones/{id}	USER/ADMIN. Listado y detalle propios.
GET /devoluciones/gestion	OPERADOR/ADMIN. Filtros y paginación.
GET /devoluciones/gestion/{id}	OPERADOR/ADMIN. Detalle de gestión.
PATCH /devoluciones/gestion/{id}/estado	OPERADOR/ADMIN. Transición e historial.
GET /productos; POST /pedidos/checkout	Catálogo público; checkout autenticado y simulado.
GET /actuator/health/readiness	Público. Estado de disponibilidad, con comprobación de BD.

Contrato de solicitud y transición

```
POST /devoluciones
{"pedidoId":1,"motivo":"OTRO","comentario":"Detalle",
"items":[{"pedidoDetalleId":1,"cantidad":1}]}

PATCH /devoluciones/gestion/1/estado
{"estadoActual":"EN_REVISION","estado":"RECHAZADA",
"observacionOperador":"Motivo de la decisión"}
```

La respuesta incorpora historial[] con estadoAnterior, estadoNuevo, actorId, actorNombre, actorRol, observacion y createdAt. Se conservan rutas propias del proyecto; los endpoints /api/... del enunciado eran sugeridos, no se presenta un prefijo inexistente.

9. Pruebas y resultados

Verificación del 15/10/2026: Maven verify: 35 pruebas, 0 fallos/errores. Angular: 4 pruebas de almacenamiento del carrito y compilación de producción correctas. Los resultados agregados se adjuntan en JSON.

Escenario requerido	Evidencia disponible
Devolución válida	DevolucionServiceTests: precio histórico, creación e identidad del primer evento.
Cantidad superior	Prueba de exceso de unidades; no se persiste una solicitud inválida.
Compra ajena	Prueba de pertenencia; la compra de otro usuario se oculta con 404.
Transición válida	Inicio de revisión y rechazo; se verifican estados, actor y observación de eventos.
Transición inválida	Salto a COMPLETADA y rechazo sin observación; no añaden eventos.
Seguridad de endpoints	MockMvc: acceso anónimo, rol cliente/operador/admin y validación de checkout.
Carrito persistente	4 casos: migración, pestaña nueva, precedencia y almacenamiento restringido.

Alcance de las pruebas

Las reglas se prueban con JUnit y Mockito. Las pruebas web MockMvc integran Spring MVC y Spring Security con servicios simulados. No se afirma disponer de Testcontainers ni de pruebas transaccionales automatizadas contra PostgreSQL real. Las capturas y consultas funcionales del despliegue complementan esas pruebas, pero no sustituyen una suite E2E.

Postman

Se adjunta una colección ampliada con autenticación, consultas, workflow y casos negativos. Durante la elaboración de este informe no se crearon compras ni devoluciones ni se cambiaron sus estados; las capturas muestran datos de prueba existentes.

10. Despliegue y configuración

Servicio	Configuración comprobada
Vercel	https://techstorepro.vercel.app/ - Angular, salida dist/frontend/browser y fallback SPA a index.html.
Render	https://techstore-pro-backend.onrender.com - servicio techstore-pro-backend, Docker, Virginia, puerto 10000.
Supabase	Proyecto inknldqrcpgwivyrjcfh; PostgreSQL vía Session pooler y esquema techstore.
Cloudinary	Subida firmada en el backend. URL segura referenciada por el producto; logo/favicon conservados en frontend.

Variables del backend (solo nombres)

SPRING_PROFILES_ACTIVE=supabase, **PORT=10000**, DB_URL, DB_USER, DB_PASSWORD, DB_POOL_SIZE, JWT_SECRET, APP_FRONTEND_ORIGIN, ADMIN_EMAIL, ADMIN_PASSWORD y CLOUDINARY_URL. OPERATOR_EMAIL/OPERATOR_PASSWORD permiten crear inicialmente un operador si aún no existe. No se muestran valores de secretos.

Construcción y controles

Dockerfile multietapa: Maven/Java 17 construye con **mvn verify**; si las pruebas fallan no se genera la versión de despliegue. El contenedor de ejecución usa Java 17, usuario sin privilegios y el puerto proporcionado por Render. Vercel usa **pnpm run verify**, que ejecuta las pruebas del carrito antes de compilar Angular.

Estado de CI y limitaciones

Hay workflows de GitHub Actions en ambos repositorios. La ejecución revisada el 13/10/2026 no inició por un bloqueo de facturación de la cuenta; no se presenta como CI exitosa. Los controles de construcción en Render/Vercel sí pasaron entonces y las pruebas locales se volvieron a ejecutar para este informe.

El plan gratuito de Render puede suspenderse por inactividad y tarda en arrancar. La imagen publicada la construye Render. La construcción local con Podman quedó pendiente de reiniciar Windows e iniciar su máquina; no se declara completada.

11. Ejecución y demostración

Repositorios separados

Backend: github.com/Fcranz28/Tech-Store-Pro, rama codex/render-backend, commit 4defc58.
Frontend: github.com/Fcranz28/Front_end_Tech_Store_Pro, main, commit c6d5ced. El backend no se presenta como integrado a main.

Instalación local

Requisitos: Java 17, Node.js 24 y pnpm compatible con el lockfile (10.28.0 usado en CI). Clonar ambos repositorios. Crear un .env privado a partir del ejemplo y configurar DB_URL/USER/PASSWORD, JWT_SECRET y las variables de bootstrap necesarias.

```
git clone -b codex/render-backend https://github.com/Fcranz28/Tech-Store-Pro.git
cd Tech-Store-Pro
.\mvnw.cmd verify
.\mvnw.cmd spring-boot:run
```

```
git clone https://github.com/Fcranz28/Front_end_Tech_Store_Pro.git
cd Front_end_Tech_Store_Pro
pnpm install --frozen-lockfile
pnpm run verify
pnpm start
```

En desarrollo, Angular usa http://localhost:8080 y sirve en http://localhost:4200. En Unix se utiliza ./mvnw. Si se apunta a la BD remota ya migrada, activar el perfil supabase; para una BD local nueva, Flyway aplica las migraciones. No ejecutar ambas historias sobre el mismo esquema sin planificación.

Acceso demo y guion de evaluación

Cliente de prueba: **checkout-qa-20261013-1706@example.invalid**
Contraseña exclusiva de la demo: **QaTechstore-13Oct2026!Flow**. No es una credencial de infraestructura ni administrativa. El administrador conserva sus credenciales por canal privado; no se imprimen en este informe.

1. Ingresar como cliente. 2. Abrir Mis pedidos y Mis devoluciones. 3. Revisar la compra entregada #1 y la solicitud ficticia #1 con tres movimientos. 4. Abrir el formulario y observar la cantidad disponible. 5. Con ADMIN, abrir gestión y probar filtros. Para demostrar aprobación/completado, usar una nueva solicitud de prueba; la rechazada no puede reabrirse.

12. Evidencias: aplicación pública



Figura 1. Inicio desplegado. Vista pública de TechStore Pro servida desde el dominio de producción.

Fuente: https://techstorepro.vercel.app/
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

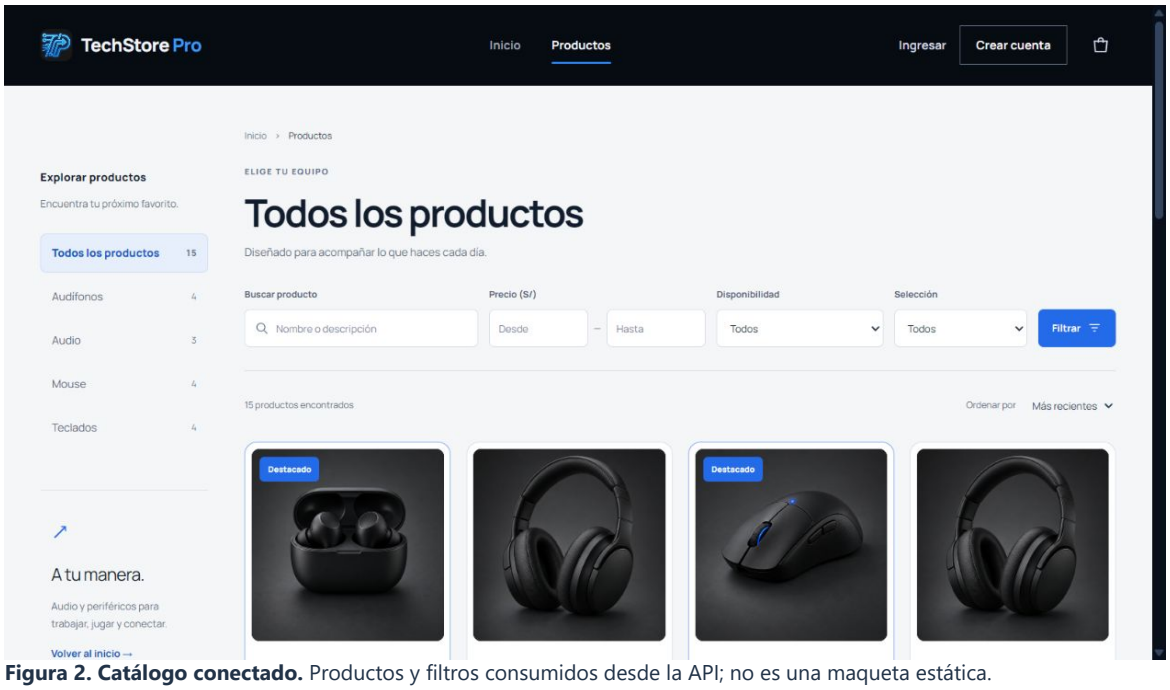


Figura 2. Catálogo conectado. Productos y filtros consumidos desde la API; no es una maqueta estática.

Fuente: https://techstorepro.vercel.app/productos
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

13. Evidencias: acceso y compras

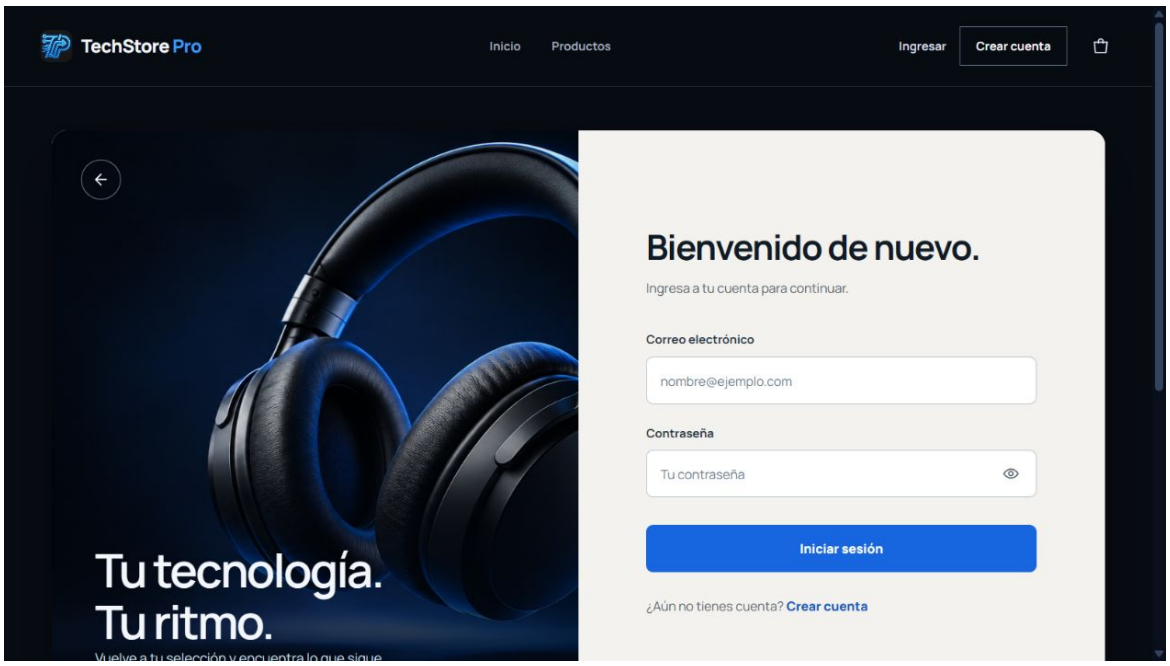


Figura 3. Autenticación. Pantalla de acceso sin mostrar contraseñas ni tokens.

Fuente: https://techstorepro.vercel.app/login
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

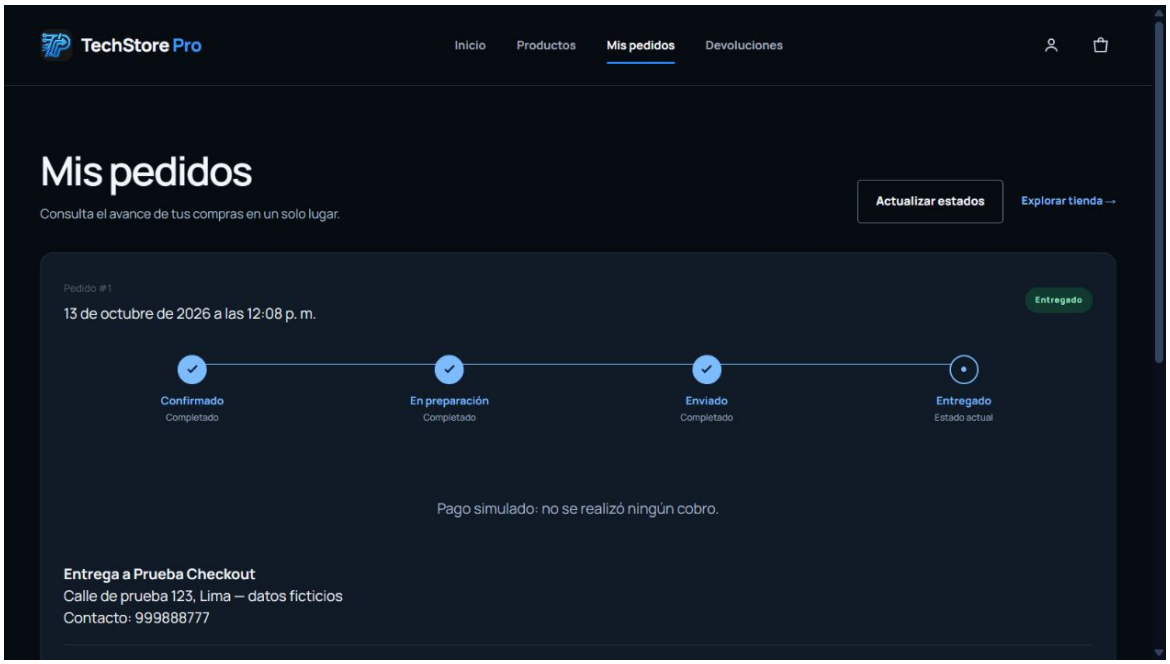


Figura 4. Compra existente. El pedido de prueba se conserva en el backend y sirve como origen de la devolución.

Fuente: https://techstorepro.vercel.app/pedidos
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

14. Evidencias: cliente y trazabilidad

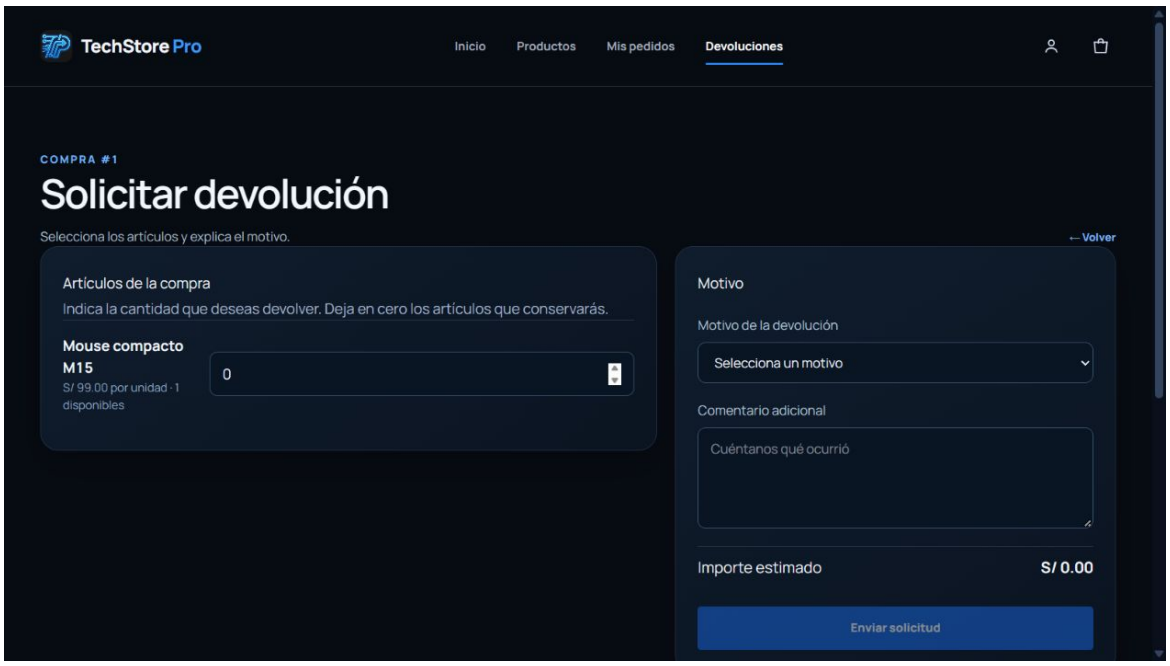


Figura 5. Solicitud reactiva. Compra, líneas, unidades disponibles, motivo y comentario. La captura no registra una solicitud nueva.

Fuente: https://techstorepro.vercel.app/devoluciones/nueva/1
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

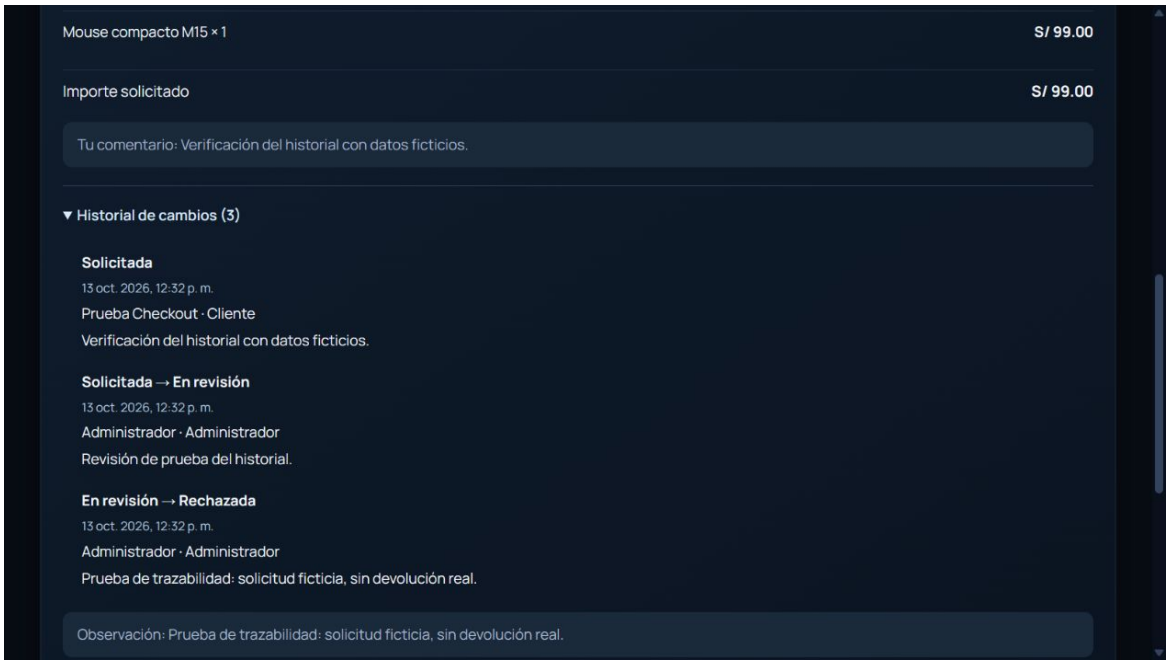


Figura 6. Historial del cliente. La solicitud ficticia muestra creación, revisión y rechazo con actor, fecha y observación.

Fuente: https://techstorepro.vercel.app/devoluciones
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

15. Evidencias: administración

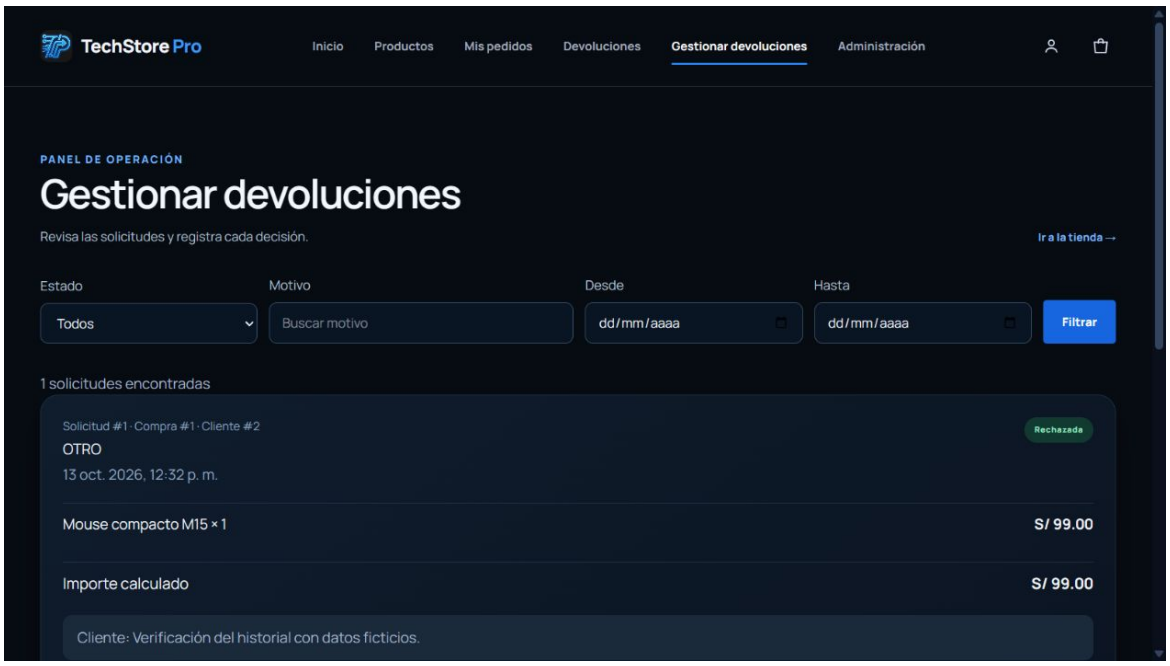


Figura 7. Gestión de devoluciones. Acceso de ADMIN, filtros y registro real de solicitudes.

Fuente: https://techstorepro.vercel.app/gestion-devoluciones
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

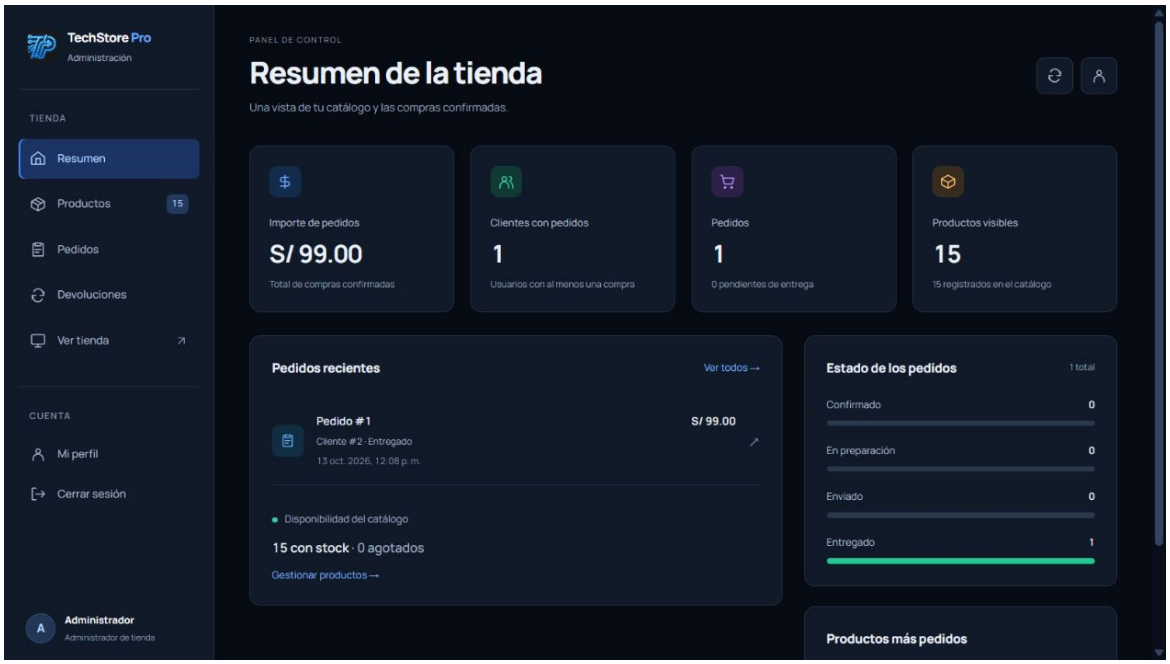


Figura 8. Administración de tienda. Resumen y herramientas de administración de los datos existentes.

Fuente: https://techstorepro.vercel.app/admin
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

16. Evidencias: disponibilidad y código

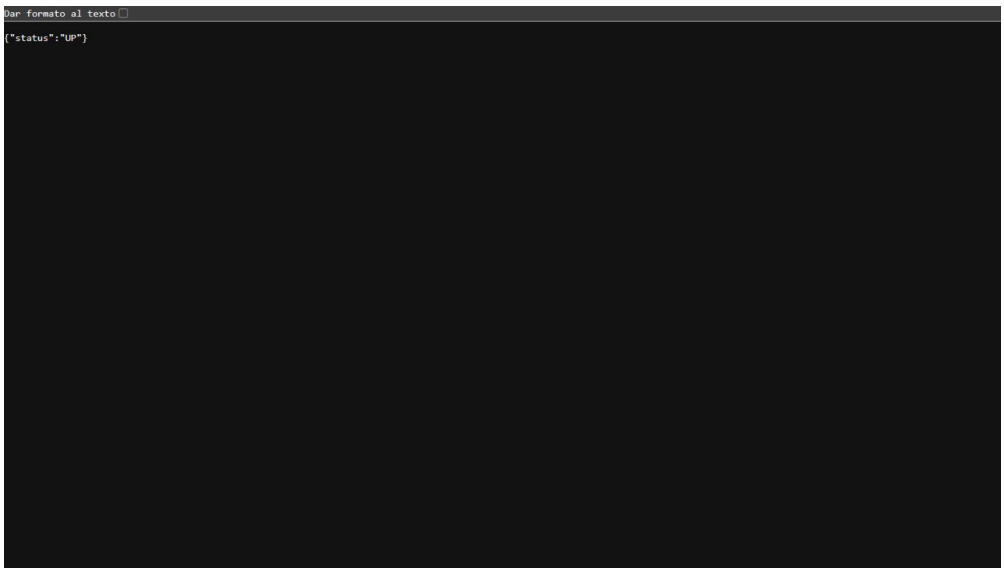


Figura 9. Backend público. Readiness del servicio de Render. El estado UP incluye la comprobación de conexión a la base de datos.

Fuente: https://techstore-pro-backend.onrender.com/actuator/health/readiness
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

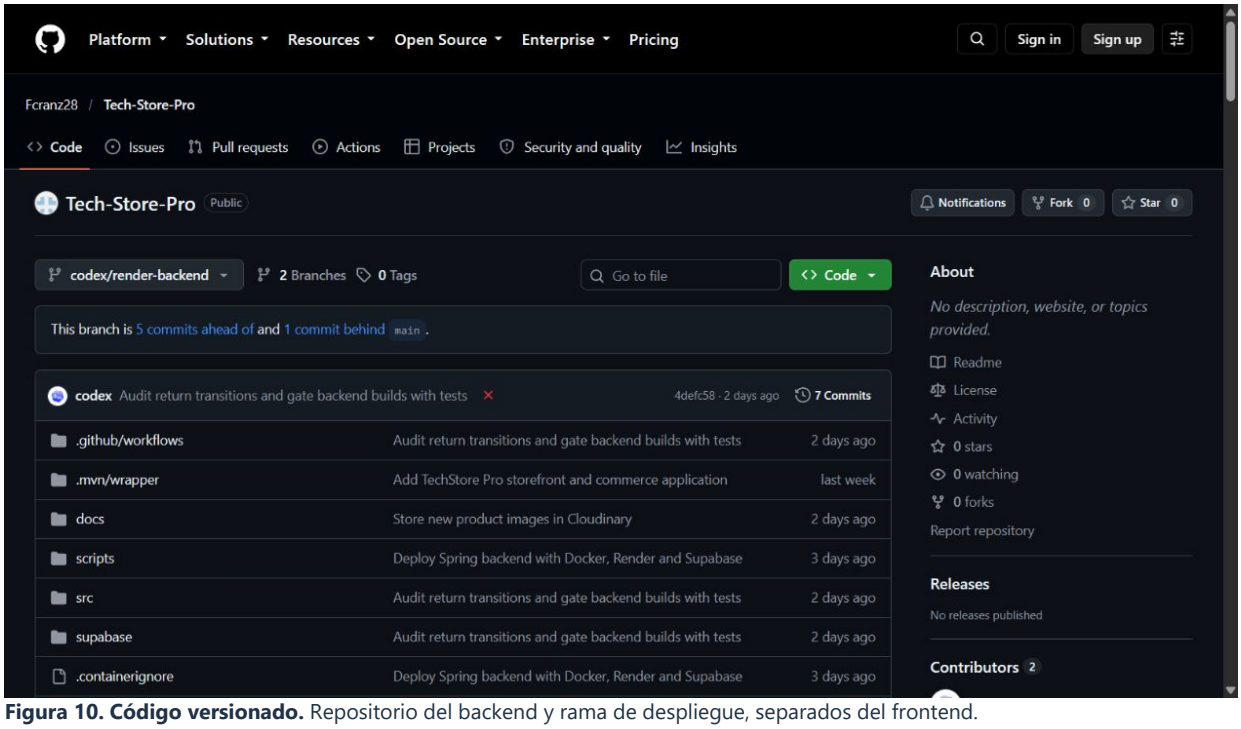


Figura 10. Código versionado. Repositorio del backend y rama de despliegue, separados del frontend.

Fuente: https://github.com/Fcranz28/Tech-Store-Pro/tree/codex/render-backend
Captura: 15/10/2026, navegador de Codex. Original disponible en los anexos.

17. Conclusiones y pendientes

Resultados

TechStore Pro integra el caso de devoluciones sin reconstruir el proyecto desde cero. Reutiliza sus compras y detalles, separa frontend/backend, aplica autorización por roles, valida pertenencia y cantidades, calcula importes con precios históricos y controla el workflow. El historial añade trazabilidad transaccional y las vistas consumen datos de la API desplegada.

Pendientes identificados

Tema	Estado / siguiente acción
GitHub Actions	Workflows publicados; resolver el bloqueo de facturación de la cuenta y volver a ejecutar los jobs.
Integración con BD real	Ampliar la suite con PostgreSQL/Testcontainers y pruebas de cantidades concurrentes por HTTP.
Postman	Importar la colección anexa y configurar usuarios/tokens propios antes de ejecutar el recorrido completo.
Podman local	Reiniciar Windows e iniciar la máquina para construir localmente; el despliegue Docker de Render sí existe.
Backend main	Las mejoras están en codex/render-backend; integrar mediante revisión cuando corresponda.
Documentación de repositorios	Mantener README alineado con el estado desplegado y versionar la colección ampliada si se incorpora al repositorio.

Fuentes y criterio de elaboración

[1] CV_DS_Tarea 3_Proyecto_Avanzado_Sistema_Devoluciones (2).pdf, 7 páginas: requisitos y entregables.
[2] SoftDevoluciones-despliegue.pdf, 8 páginas: referencia de estructura de informe; no se copian nombres, URLs o funciones como si fueran del proyecto.
[3] Repositorios, configuración, pruebas y migraciones de TechStore Pro.
[4] Capturas reales del navegador obtenidas el 15/10/2026; manifiesto de URL y fecha en el anexo.

Las capturas acreditan la interfaz y el acceso observados, no garantizan disponibilidad permanente. La evidencia de CI bloqueada y las limitaciones de las pruebas se mantienen explícitas para evitar atribuir resultados no ejecutados.